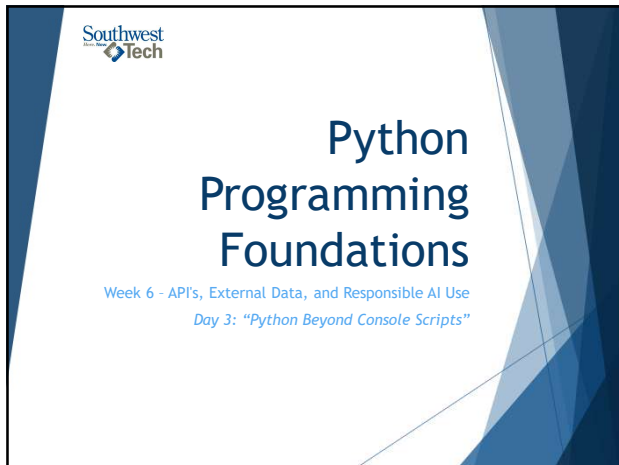
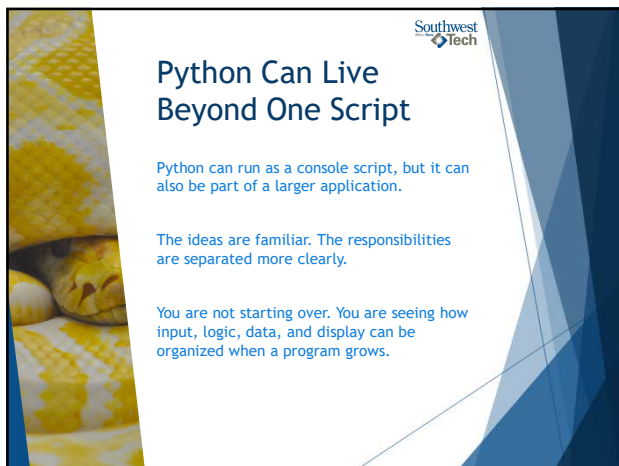




1



2



3



Review: API Output Still Needs Explanation

28 32 14

REVIEW

Click here for more information.

CLICK 1

For Assignment 11, the goal is not just to retrieve data.

The goal is to explain where the data came from, what shape it had, and which values your program selected.

That same explanation habit matters in larger applications: know where information enters, how it is handled, and what result is shown.

4



A11 Closeout

1. Data Source	2. Selected Values	3. Explanation
Where the data came from.	The useful data extracted.	The path and why it matters.
 or  weather_api.py used simulated_weather_response.json	City: Madison Temp: 72 Condition: Clear	Source: simulated file simulated_weather_response.json ↓ Path: data → current → temperature, weather ↓ Why: Retrieves current weather for a city so the user can see accurate info.

 Retrieving data is not enough; explain the path.

Review: API Output Still Needs Explanation

5



What Counts as Success Today

The main points:

Identify where input enters	Identify where validation or request handling happens	Identify where logic lives	Identify where output/display happens	Explain how a larger app separates responsibility
-----------------------------	---	----------------------------	---------------------------------------	---

↓

This is recognition first.
You are not building a Django app today.

↓

Your task is to point to the parts and explain their jobs in plain language.

6

Recognize Larger App Architecture
Same ideas. More structure. Clear responsibilities.

Small Console Script
One file handles everything. Great place to start.

```

# mirror_script.py
# get input
# check input
# do the work
# show result

```

All responsibilities live in one file.

Recognition Path (5 Steps)

- 1 Input**: Get data from the user or a source.
- 2 Validation**: Check and prepare the data.
- 3 Logic**: Process the data and apply rules.
- 4 Output / Display**: Show clear results to the user.
- 5 Separated Responsibilities**: Each part has its own job and place.

Larger App Flow
Responsibilities live in separate parts.

```

graph TD
    input[Input: input_mirror.py] --> validation[Validation: validation_mirror.py]
    validation --> logic[Logic: processor.py]
    logic --> output[Output / Display: display.py]
    
```

Separated responsibilities allow focused modules.

★ **Recognition first. No Django build today.**
Name where responsibilities live so you can grow with confidence.

Today's Success

7

What We Will Use Today

- ▶ input
- ▶ validation
- ▶ logic
- ▶ display
- ▶ template
- ▶ view
- ▶ model as recognition vocabulary

These words help us talk about responsibility. They are not a requirement to build a full framework project.

8

Today's Architecture Vocabulary
Seven building blocks you will see in larger applications.

ARCHITECTURE TOOLBOX

input
Where data enters the application.

validation
Checks and prepares data for safe use.

logic
Applies rules and produces results.

display
Shows information to the user.

template
The layout used to present information.

view
Handles requests and connects components.

model
Represents and manages the data.

★ You don't need to build these today—just **recognize** where each responsibility lives.

Toolbox

9

Southwest
Tech

What We Will “Skip” For Now

We will save these for later:

- ▶ installing Django
- ▶ building a full web app
- ▶ databases
- ▶ authentication
- ▶ deployment

Today we inspect the shape of a larger app, not build one.

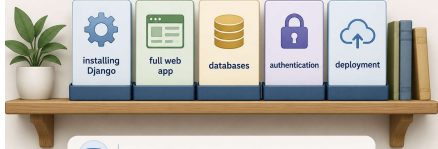


10

Southwest
Tech

Not Today's Build

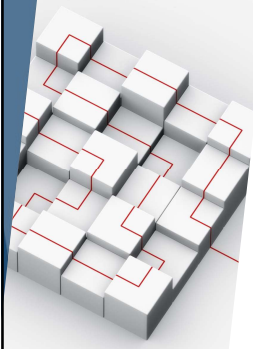
Save for Later



Today we inspect the shape of a larger app.

Parked For Later

11

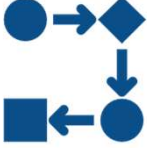


Larger Application Structure

12

Southwest
Tech

Larger Apps Separate Responsibilities



As programs grow, it becomes useful to separate responsibilities.

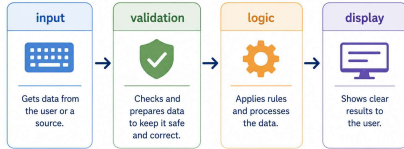
Different parts of the application can handle input, validation, data logic, and display.

Separation makes the app easier to read, change, test, and explain because each part has a clearer job.

13

Southwest
Tech

Larger Apps Separate Responsibilities



Each part has a clearer job.

Larger Apps Separate Responsibilities

14

Southwest
Tech

Input Enters Through A Controlled Place

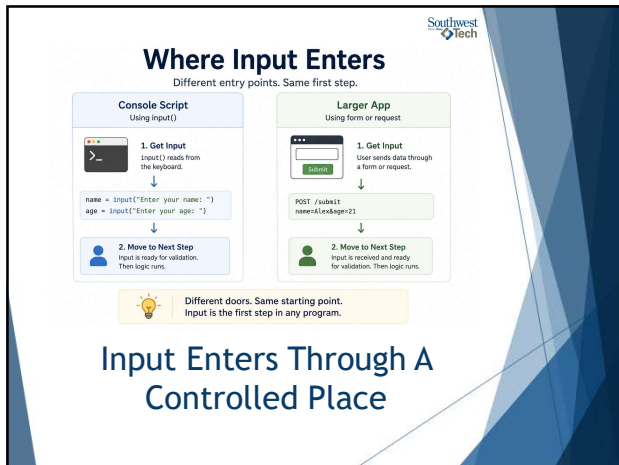


In a console script, input may come from `'input()'`.

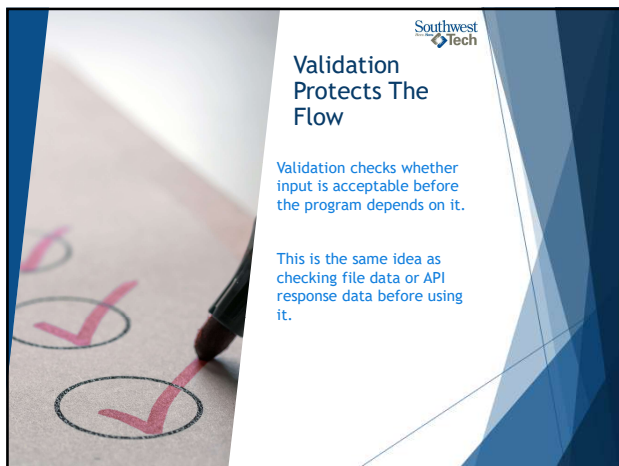
In a larger app, input may come through a form, request, or controlled interface.

The source changes, but the question stays familiar: what information is coming in, and what should the program do with it?

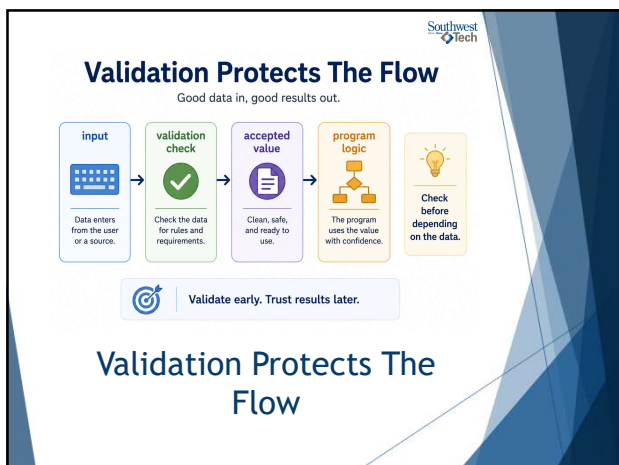
15



16



17



18



MVT As Recognition Vocabulary

MVT is one way to talk about separated responsibilities:

- Model: data shape and rules
- View: request handling and decisions
- Template: displayed result

Today you only need to recognize the flow.

You may also hear the term MVC. The names shift across frameworks, but the shared idea is the same: separate data, logic, and display.

19



MVT Recognition: How Data Becomes a Page

Three parts with clear jobs working together.

Model

data shape and rules

Defines the data structures and the rules about the data.



→

View

request handling and decisions

Receives the request, checks/validates, gets data, and decides what to show.



→

Template

displayed result

Takes the data from the view and renders the final output for the user.





Big idea:
Models hold data and rules. Views handle requests and decisions. Templates turn data into what the user sees.

MVT As Recognition Vocabulary

20



MVC And MVT Use Different Names

You may also hear MVC: Model, View, Controller.

Django uses MVT: Model, View, Template.

The names are not identical:

- MVC View is closer to what the user sees.
- Django Template is what the user sees.
- Django View handles request decisions.

The shared idea is responsibility separation.

21



Full Stack As A Preview

A full-stack application connects multiple parts:

- ▶ what the user sees
- ▶ the logic that handles requests
- ▶ the data the app stores or retrieves
- ▶ the flow between those parts

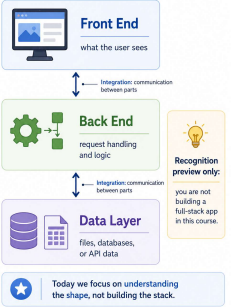
In this course, you only need to recognize the idea. You are *not expected to build a full-stack app.*

22



Full Stack As A Preview

A big picture view of how applications are organized.



Front End
what the user sees

Back End
request handling and logic

Data Layer
files, databases, or API data

Recognition preview only:
you are not building a full-stack app in this course.

Today we focus on understanding the shape, not building the stack.

23



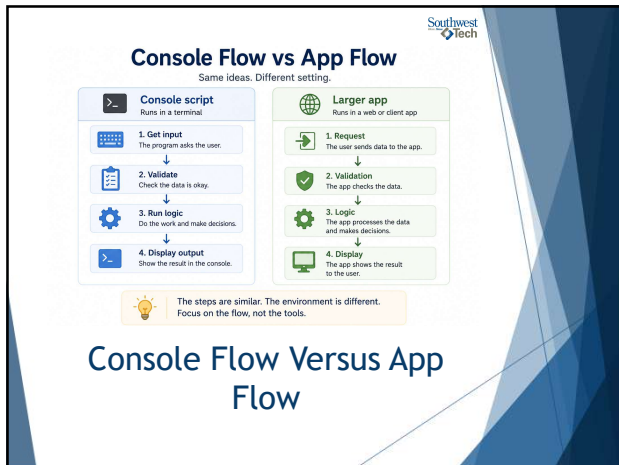
Console Flow Versus App Flow

A console script may run from top to bottom.

A larger app often responds to a request, validates input, runs logic, and then returns a display.

That means the flow may be request-driven instead of simply line-by-line from the top of one file.

24



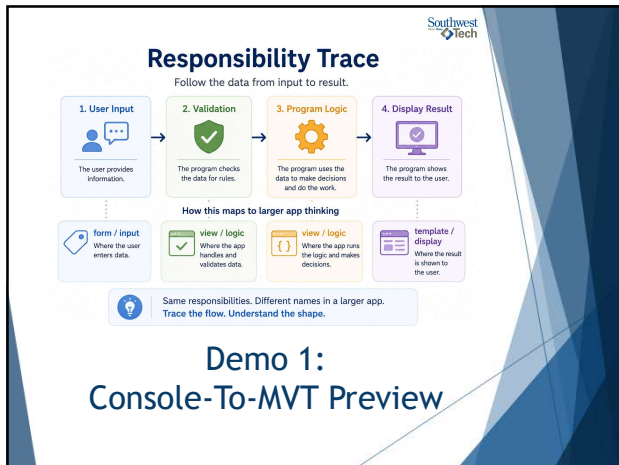
25



26



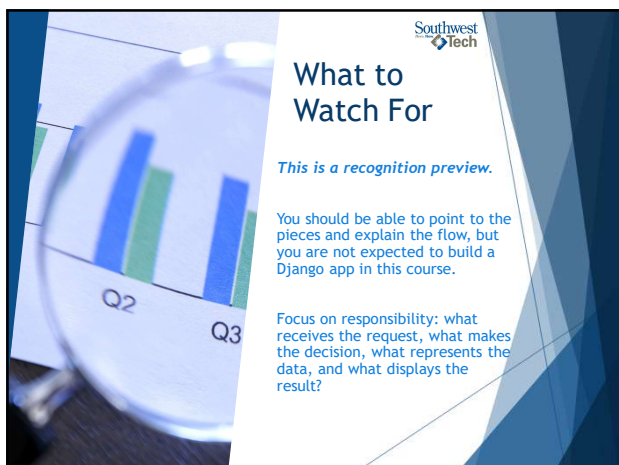
27



28



29



30

Southwest
Tech

MVT Recognition Preview

Three parts. Clear jobs. Working together.

MODEL
data shape



Defines the data structures and rules.

↔

VIEW
request handling



Receive the request, processes data, and decides what to do.

↔

TEMPLATE
display



Turns the result into output for the user.



Recognize the flow; do not build Django today.

Demo 2: Django MVT Recognition

31

Southwest
Tech

Common Failure: Preview Becomes Panic



Seeing a larger structure does not mean you are behind.



Recognition comes before implementation. Today, naming the parts and explaining the flow is the success target.



This preview is meant to make later courses less surprising, not to create a new hidden requirement.



DON'T Panic

32

Southwest
Tech

Preview Is Not Panic

You are building understanding, not the whole system.

Recognition today

Your goal is to see the shape.

- ✓ Learn the parts and their jobs.
- ✓ See how they connect.
- ✓ Understand the flow.

Focus: awareness and clarity.

Implementation later

Your goal is to build with confidence.

- ✓ Apply the parts in a real project.
- ✓ Make good design choices.
- ✓ Iterate and improve over time.

Focus: practice and growth.



Naming the parts is the success target.
Today: recognize. Later: implement.

Common Failure: Preview Becomes Panic

33



Southwest
Tech

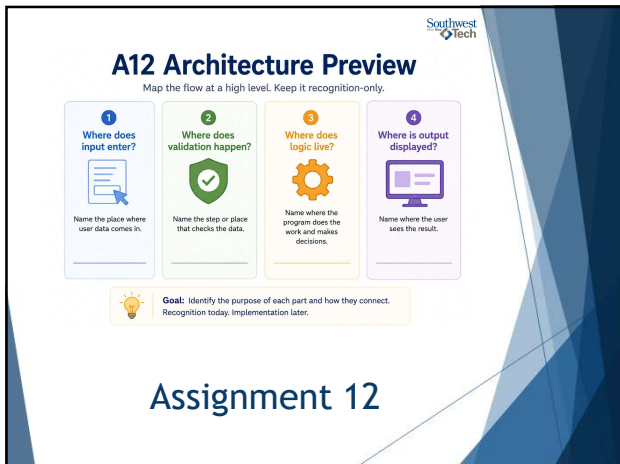
Assignment #12

For Assignment 12: Inspect a guided Python app-architecture example.

Identify:

- ▶ where input enters
- ▶ where validation happens
- ▶ where logic lives
- ▶ where output is displayed

34



Southwest
Tech

A12 Architecture Preview

Map the flow at a high level. Keep it recognition-only.

1

Where does input enter?



Name the place where user data comes in.

2

Where does validation happen?



Name the step or place that checks the data.

3

Where does logic live?



Name where the program does the work and makes decisions.

4

Where is output displayed?

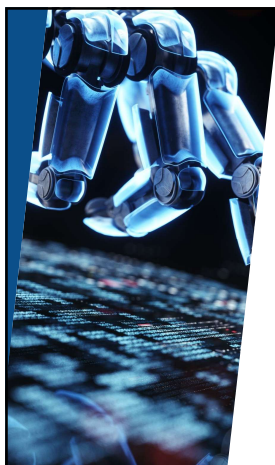


Name where the user sees the result.

 **Goal:** Identify the purpose of each part and how they connect. Recognition today, implementation later.

Assignment 12

35

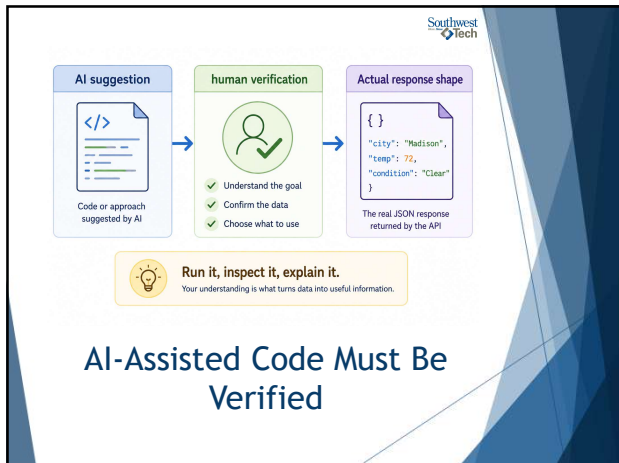


Southwest
Tech

AI Use In Lab 0n

- ▶ AI can suggest API code that looks believable but does not match the actual response.
- ▶ You must inspect the response, run the code, and explain what human decision you made.
- ▶ If AI names a field that does not exist in your response, your program is still wrong even if the code looks professional.

36



37

Southwest Tech

Evidence For A11 & A12

- ▶ For A11, show selected API-style output and explain your data path.
- ▶ For A12, explain the larger app flow in plain language and identify the main parts.
- ▶ Include an AI-use note if AI helped explain code or vocabulary.

38

Southwest Tech

Evidence Snapshot

Show what you understand. Keep it simple.

A11	A12
Data Path Show the path the data takes from start to finish.	Input Location Where does input enter your application?
Selected Output Show one clear output from your program.	Logic Location Where does your program do the work and decisions?
Validation Note Note how you checked the data before using it.	Display Location Where is the result shown to the user?

Explain the flow in your own words.
 Write a short explanation that connects the parts.

Evidence For A11 & A12

39



Closing Success Check

By the end of today, you should be able to say:

"This part receives input, this part checks or handles it, this part runs the logic, and this part displays the result."



40



Thank you!

Have Fun!

41
